

Security Advisory (Draft): Domain Allowlist/Denylist Bypass via Unbounded String-Prefix Match

Status: drafted for review, not yet published as a GitHub Security Advisory and no CVE has been requested or assigned. Publishing it for real is a repository-owner action (see §7) — this document is the complete content for that form, not a substitute for filing it.

1. Summary

`SecurityWatchdog._is_url_match()` (`browser_use/browser/watchdogs/security_watchdog.py`) matched a "full URL" allowlist/denylist pattern (e.g. `allowed_domains=['https://example.com']`) against the navigated URL using an unbounded string-prefix comparison (`url.startswith(pattern)`). Because `"https://example.com"` is a literal string prefix of `"https://example.com.evil.com"` , an attacker-controlled host sharing that prefix — with no DNS label boundary — was accepted as if it were the allowed domain. The fix (this repository, commit `ed4bf31`) parses the pattern and requires an exact scheme+host match before falling back to a path-prefix check under the already-verified host.

2. Classification

- **CWE-697: Incorrect Comparison** — a string-suffix/prefix test is used where a hostname-boundary comparison is required.
- **CWE-284: Improper Access Control** (secondary) — the domain-restriction feature exists specifically to be an access-control boundary; the defect defeats that boundary's intended guarantee.

This is a well-documented bug class, not specific to this codebase — the same root cause (allowlist/denylist logic implemented as `startswith()` or `endswith()` without an explicit label-boundary check) appears repeatedly across unrelated projects and is the subject of dedicated vulnerability advisories and detection rules elsewhere (see §6, references).

3. CVSS v3.1 vector (estimated, not an authoritative assignment)

```
AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
```

- **AV:N** — reachable over the network (the attacker only needs the agent to navigate to a URL under their control, e.g. via a same-site open redirect, a compromised advertisement, or a crafted link in page content the agent is asked to follow).

- **AC:L** — once an attacker controls a domain sharing the allowed pattern as a string prefix (fully within their own control — domain registration, not a race condition or other hard-to-hit precondition), exploitation is direct.
- **PR:N** — no privilege on the target system is required.
- **UI:R** — requires the agent to navigate to the attacker's URL (as part of its own task, following a link, or via a redirect).
- **S:U** — the bypass stays within the browsing-session security context; it does not itself grant access to a different security authority (e.g. host OS, a different VM).
- **C:L / I:L** — direct impact is confined to what happens once the agent is on the unintended domain (a prompt-injection surface, potential session/credential exposure if the agent carries authenticated state); this advisory does not assume a specific downstream action, so impact is scored conservatively rather than assuming worst case.
- **A:N** — no availability impact.

Recompute this precisely with the official calculator before publishing:

<https://www.first.org/cvss/calculator/3.1> — qualitatively this sits in the Medium range; do not take the vector string above as a substitute for running it through the calculator, since the exact score depends on severity judgment calls (particularly C/I) that deserve a second opinion.

4. Affected component

- File: `browser_use/browser/watchdogs/security_watchdog.py`
- Function: `SecurityWatchdog._is_url_match()`
- Affected configuration: any `BrowserProfile` using a **full-URL** pattern (containing `://`) in `allowed_domains` or `prohibited_domains` — e.g. `allowed_domains=['https://example.com']`. Domain-only patterns (no scheme, e.g. `allowed_domains=['example.com']`) and glob patterns (`*.example.com`) use different, unaffected code paths in the same file.

5. Proof of concept

```
# Unfixed code (pre-ed4bf31):
allowed_domains = ['https://example.com']
# The following URLs were both incorrectly treated as allowed:
_is_url_allowed('https://example.com.evil.com/path') # -> True (should be False)
_is_url_allowed('https://example.comevil.com/path')   # -> True (should be False)
# Same-host, deeper-path continuation correctly remained allowed both before and after the fix:
_is_url_allowed('https://example.com/some/deeper/path') # -> True (correct, unaffected)
```

Full reproduction, including the regression test that fails on unfixed code and passes on fixed code, is `tests/ci/test_security_watchdog_full_url_pattern_host_boundary.py` in this repository.

6. Fix

Commit `ed4bf31` (this repository). Parses the pattern with `urlparse` and requires the incoming URL's scheme and hostname to match the pattern's scheme/hostname *exactly* (case-insensitive) before permitting a path-prefix check under that verified host. No change to the glob-pattern or domain-only-pattern branches, which already performed host-exact rather than string-prefix comparisons.

References for the bug class (independent of this codebase)

- Real-world instances of the identical root cause (surfaced during classification research for this advisory, not claimed as related incidents to this project): a hostname allowlist bypass via suffix over-match tracked as CWE-697/CWE-284 in another open-source proxy project — <https://github.com/tinyproxy/tinyproxy/issues/627>
- A webhook URL allowlist bypassed via `startswith / endsuffix` on a subdomain or suffix — <https://github.com/ApexChainx/ApexChainx-Backend/issues/463>
- General treatment of allowlist authority-bypass patterns for non-IP internal hostnames — <https://agentthreatrule.org/en/rules/ATR-2026-02107>

7. How to actually publish this (repository-owner action)

This document is written to be pasted directly into GitHub's own advisory form. The repository owner (not this session) needs to do it, because it requires repository admin access:

1. Go to the repository's **Security** tab -> **Advisories** -> **New draft security advisory**. Docs: <https://docs.github.com/en/code-security/concepts/vulnerability-reporting-and-management/repository-security-advisories>
2. Fill the form using §1 (summary), §2 (CWE), §4 (affected component/ versions), §5 (PoC) from this document.
3. From that draft advisory, click **Request CVE ID** — GitHub acts as a CVE Numbering Authority and typically responds within about 72 hours; requesting a CVE does not itself make the advisory public. Docs: <https://docs.github.com/en/code-security/security-advisories/working-with-repository-security-advisories/publishing-a-new-security-advisory>
4. Only once the fix (commit `ed4bf31`, already merged) is live in a released version should the advisory be published — GitHub publishes the CVE record automatically once the advisory itself is made public.

Note on 2026 processing volume: GitHub's advisory database has been running well above its historical throughput this year (over 1,500 advisories published in a single month per their own reporting), so expect review latency measured in weeks, not days, for the human-reviewed steps.